ПЗ 3 Модели ML в Dask

Теоретическая часть

Традиционные библиотеки машинного обучения, такие как Scikit-learn, отлично работают с данными, которые помещаются в оперативную память (RAM) одного компьютера. Однако, когда объем данных превышает доступную память (так называемые "out-of-core" или "out-of-memory" данные), или когда вычисления требуют распределения по нескольким ядрам/процессорам, Scikit-learn сталкивается с ограничениями. Dask решает эту проблему. Это гибкая библиотека для параллельных вычислений в Python, которая позволяет масштабировать ваши вычисления и анализ данных с одного компьютера с множеством ядер на кластеры из множества серверов. Dask не заменяет Scikit-learn, а расширяет его, предоставляя инструменты для работы с большими данными, используя знакомый API.

Ключевые концепции Dask, важные для ML.

Dask-массивы (`dask.array`): ведут себя как массивы NumPy, но разбиты на множество маленьких "чанков" (блоков). Эти чанки могут обрабатываться параллельно и независимо друг от друга, возможно, даже на разных машинах кластера. Операции над всем массивом автоматически разбиваются на операции над этими чанками.

Dask-датафреймы (`dask.dataframe`): аналогичны Pandas DataFrame, но также разделены на партиции (части). Позволяют работать с данными, которые не помещаются в память, выполняя операции (фильтрация, группировка, агрегация) блоками.

Граф вычислений (Task Graph): Dask не выполняет вычисления сразу. Вместо этого он строит ориентированный ациклический граф (DAG), который представляет собой план всех необходимых операций и их зависимостей. Этот граф оптимизируется и затем выполняется планировщиком (scheduler) максимально параллельным и эффективным способом.

Планировщики (Schedulers): Dask имеет несколько планировщиков для выполнения графа вычислений:

hreaded/Multiprocessing: Для параллельной работы на одном компьютере.

Distributed: Для работы на кластере. Это самый мощный и рекомендуемый планировщик, предоставляющий диагностическую панель в реальном времени.

Подходы к реализации ML в Dask

Dask предлагает три основных стратегии для машинного обучения, каждая со своей нишей применения.

Параллельная обработка с помощью `dask_ml.joblib`. Этот подход позволяет распараллелить существующие модели Scikit-learn на множестве ядер одного компьютера. Вы используете привычный код Scikit-learn, но оборачиваете его вызов в `parallel_backend('dask')`. Dask берет на себя распределение процессов (например, подбор гиперпараметров с

`GridSearchCV` или обучение ансамблевых моделей) по ядрам CPU, эффективно управляя памятью и задачами. Применяется, когда ваши данные помещаются в память, но само обучение модели (особенно перебор параметров) требует много времени и его можно ускорить за счет параллелизма.

Алгоритмы, предназначенные для больших данных (`dask_ml`). Dask-ML – это собственная библиотека, которая предоставляет реализации алгоритмов ML, изначально спроектированных для работы с блоками данных (chunk-by-chunk). Эти алгоритмы не пытаются загрузить все данные в память сразу.

Примеры моделей:

Линейные модели (линейная регрессия, логистическая регрессия, Lasso, Ridge) с использованием алгоритмов стохастического градиентного спуска (SGD) или обычного метода наименьших квадратов.

Кластеризация (K-Means, DBSCAN).

Предобработка данных (StandardScaler, OneHotEncoder, PCA), которые могут обрабатывать данные по частям.

Ансамбли (например, `RandomForestClassifier`), которые обучают каждое дерево независимо и параллельно. Также применяются, когда ваши данные не помещаются в оперативную память и их необходимо обрабатывать по частям. Это основная причина использования Dask для ML.

Инкрементное/онлайн обучение (Partial Fitting). Многие алгоритмы в Scikit-learn и Dask-ML поддерживают метод `.partial_fit()`. Это позволяет обучать модель не сразу на всех данных, а постепенно, небольшими «пакетами» (batches). Вы последовательно загружаете данные кусками (например, по 100 000 строк), и для каждого куска вызываете `model.partial_fit(batch)`. Модель обновляет свои веса, учитывая новые данные. Это эффективно с точки зрения памяти и отлично подходит для потоковых данных. Используется для обработки потоковых данных или когда dataset настолько велик, что даже его загрузка по чанкам для полного обучения (`.fit()`) затруднена.

Типовой пайплайн ML в Dask.

1. Загрузка данных: Данные считываются не сразу, а лениво, с созданием Dask DataFrame или Array. Указывается размер чанка.

2. Предобработка и feature engineering: Все операции (заполнение пропусков, кодирование категориальных переменных, масштабирование) выполняются с помощью методов Dask, которые строят граф вычислений, но не выполняют их немедленно.

3. Разделение на train/test: используется `dask_ml.model_selection.train_test_split`, который работает с чанками.

4. Обучение модели: выбор одного из трех описанных выше подходов. Если используется Dask-ML, вызывается `model.fit(X_train, y_train)`. Граф вычислений выполняется, и модель обучается.

5. Предсказание и оценка: использование обученной модели для предсказания на тестовом наборе (`model.predict(X_test)`) и расчет метрик.

Возможности:

Масштабируемость: позволяет работать с данными, превышающими объем оперативной памяти.

Параллелизм: автоматическое распараллеливание вычислений на CPU.

Интеграция с экосистемой Python: знакомый интерфейс, похожий на Pandas, NumPy и Scikit-learn.

Гибкость: Работа как на одном компьютере, так и на распределенном кластере.

Ограничения:

Наложение расходов (Overhead): построение графа задач и его планирование создают дополнительную нагрузку. Для маленьких данных Scikit-learn будет значительно быстрее.

Не все алгоритмы реализованы: далеко не все модели Scikit-learn имеют свои полные аналоги в Dask-ML. Сложные алгоритмы, такие как Gradient Boosting (XGBoost, LightGBM), часто имеют свои собственные встроенные методы распараллеливания, которые могут быть эффективнее.

Сложность отладки: работа с распределенными системами всегда сложнее, чем с однопоточными приложениями. Могут возникать специфические ошибки.

Вот код, демонстрирующий построение модели машинного обучения с использованием библиотеки Dask для работы с большими данными:

Методические рекомендации:

Для каждого варианта имеется файл Excel «логистическая_регрессия_данные» с признаками, влияющими на принятие решения «выдать кредит клиенту» или «не выдать», а также результативной переменной бинарного характера, указывающее на результат решения по кредиту.

Требуется, используя библиотеку dask j,exbnm логистическую модель ML и сделать прогноз.

Пример реализации:

```
#импортируем библиотеки

import dask.array as da

import dask.dataframe as dd

import dask_ml.model_selection as dcv

import dask_ml.preprocessing as dpre
```

```
import dask_ml.linear_model as dlm
from dask_ml.datasets import make_classification
from dask.distributed import Client
import matplotlib.pyplot as plt
import numpy as np
import time
```

Далее необходимо инициализировать Dask клиента для распределенных вычислений.

```
# Инициализация Dask клиента для распределенных вычислений
def setup_dask_cluster():
    """Настройка локального Dask кластера"""
    client = Client(n_workers=4, threads_per_worker=2, memory_limit='2GB')
    print("Dask Dashboard доступен по адресу:", client.dashboard_link)
    return client
```

Создание клиента для управления распределенными вычислениями:

n_workers=4 - создает 4 рабочих процесса (отдельные Python-процессы)

threads_per_worker=2 - каждый worker использует 2 потока для параллелизма

memory_limit='2GB' - ограничение памяти на каждый worker (2 ГБ)

Dashboard предоставляет веб-интерфейс для мониторинга прогресса вычислений

Далее необходимо получить данные. Сначала возможно использование синтетических данных. После этого необходимо использовать данные по своему варианту из Excel-файла «логистическая_регрессия_данные».

```
# Генерация синтетических данных
def generate_data():
    """Генерация большого набора данных для демонстрации"""
```

```
    print("Генерация данных...")

    # Создаем большой dataset с помощью Dask
    X, y = make_classification(n_samples=100000, n_features=20,
                               chunks=10000, random_state=42)

    # Преобразуем в Dask DataFrame для удобства
    X_df = dd.from_dask_array(X, columns=[f'feature_{i}' for i in range(20)])
    y_df = dd.from_dask_array(y, columns=['target'])

    print(f"Размер данных: {X_df.shape[0].compute()} строк, {X_df.shape[1].compute()} столбцов")
    return X_df, y_df
```

Далее проведем предобработку данных и масштабирование признаков.

```
# Предобработка данных
def preprocess_data(X, y):
    """Предобработка данных с использованием Dask ML"""
    print("Предобработка данных...")

    # Масштабирование признаков
    scaler = dpre.StandardScaler()
    X_scaled = scaler.fit_transform(X)

    # Масштабирование признаков: StandardScaler стандартизирует признаки
    # путем удаления среднего значения и масштабирования до единичной дисперсии.
    # Это важно для многих ML-алгоритмов, чувствительных к масштабу данных.
```

Dask обрабатывает данные частями, что позволяет работать с большими данными,

не загружая их полностью в память.

Затем разделим датасет на тестовую и тренировочную части.

```
def prepare_data_dask(X, y):
    """Подготовка и разделение данных на train/test с использованием Dask"""

    # Масштабирование признаков
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    # Разделение на train/test с Dask
    X_train, X_test, y_train, y_test = dcv.train_test_split(X_scaled, y, test_size=0.2, random_state=42)

    return X_train, X_test, y_train, y_test, scaler
```

Затем обучаем модель. Код ниже обучает модель логистической регрессии с использованием библиотеки Dask ML.

- Использует L2-регуляризацию (Ridge) для предотвращения переобучения
- Применяет оптимизатор L-BFGS для эффективного обучения
- Замеряет и возвращает время обучения
- Устанавливает random_state для воспроизводимости результатов
- `C=1.0` - стандартная сила регуляризации
- `max_iter=1000` - достаточное количество итераций для сходимости

```
# Обучение модели
def train_model(X_train, y_train):
    """Обучение логистической регрессии с использованием Dask"""
    print("Обучение модели...")
```

```
    # Создаем и обучаем модель
    model = dlm.LogisticRegression(
        penalty='l2',
        C=1.0,
        solver='lbfgs',
        max_iter=1000,
        random_state=42
    )

    start_time = time.time()
    model.fit(X_train, y_train)
    training_time = time.time() - start_time

    print(f"Время обучения: {training_time:.2f} секунд")
    return model, training_time



# Кросс-валидация и подбор гиперпараметров
def hyperparameter_tuning(X_train, y_train):
    """Подбор гиперпараметров с помощью GridSearchCV"""
    print("Подбор гиперпараметров...")

    # Определяем параметры для поиска
    param_grid = {
        'C': [0.1, 1.0, 10.0],
        'penalty': ['l2', 'none']
    }
```

```
    # Создаем модель для кросс-валидации
    model = dlm.LogisticRegression(solver='lbfgs', max_iter=1000, random_state=42)

    # GridSearch с использованием Dask
    grid_search = dcv.GridSearchCV(
        model, param_grid, cv=3, scoring='accuracy'
    )

    start_time = time.time()
    grid_search.fit(X_train, y_train)
    tuning_time = time.time() - start_time

    print(f"Лучшие параметры: {grid_search.best_params_}")
    print(f"Лучшая точность: {grid_search.best_score_:.4f}")
    print(f"Время подбора параметров: {tuning_time:.2f} секунд")

    return grid_search.best_estimator_, tuning_time

# Оценка модели
def evaluate_model(model, X_test, y_test):
    """Оценка качества модели"""
    print("Оценка модели...")

    # Предсказания
    y_pred = model.predict(X_test)
    y_pred_proba = model.predict_proba(X_test)

    # Вычисляем accuracy
```

```
    accuracy = da.mean(y_pred == y_test).compute()
    print(f"Accuracy на тестовой выборке: {accuracy:.4f}")

    return accuracy

# Визуализация результатов
def visualize_results(accuracy, training_time, tuning_time=None):
    """Визуализация результатов работы"""
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

    # График точности
    ax1.bar(['Accuracy'], [accuracy], color='skyblue')
    ax1.set_ylabel('Accuracy')
    ax1.set_title('Точность модели')
    ax1.set_ylim(0, 1)

    # Добавляем значения на столбцы
    for i, v in enumerate([accuracy]):
        ax1.text(i, v + 0.01, f'{v:.4f}', ha='center', va='bottom')

    # График времени выполнения
    times = [training_time]
    labels = ['Обучение']
    if tuning_time:
        times.append(tuning_time)
        labels.append('Подбор параметров')

    ax2.bar(labels, times, color='lightcoral')
    ax2.set_ylabel('Время (секунды)')
```

```
ax2.set_title('Время выполнения')


plt.tight_layout()
plt.show()
```

Вопросы для самоконтроля

1. Почему в функции `generate_data()` параметр `chunks=10000` критически важен при работе с Dask, и какие проблемы могли бы возникнуть, если бы он не был указан или был бы слишком большим/маленьким?
2. Чем принципиально отличается вызов `scaler.fit_transform(X)` в Dask от аналогичного вызова в Scikit-learn, с точки зрения обработки данных и использования памяти, особенно когда `X` – это Dask DataFrame?
3. Какое ключевое преимущество дает использование `dcv.train_test_split` вместо `sklearn.model_selection.train_test_split` в контексте этого пайплайна, учитывая природу исходных данных (`X_scaled`, `y`)?
4. Почему для логистической регрессии в `train_model()` и `hyperparameter_tuning()` выбран именно решатель `solver='lbfgs'`, и какие ограничения или особенности Dask могли повлиять на этот выбор (например, по сравнению с более привычным `sag` или `saga` в Scikit-learn)?
5. Как функция `evaluate_model()` обходит ограничения Dask при вычислении accuracy с помощью конструкции `da.mean(y_pred == y_test).compute()`? Почему просто вызвать `.mean()` было бы недостаточно, и что именно происходит в момент вывода результата на экран?